

Parallelizing A* Pathfinding on a 2D Grid for Game AI

Hoang Nguyen
Computer Science Department
San Jose State University
San Jose, CA 95192

ABSTRACT

Pathfinding is one the core aspects of artificial intelligence in video games, enabling NPCs to navigate their environments. Finding an optimal path efficiently is critical, as search performance directly affects frame rate and player experience. This challenge grows significantly when there are multiple agents. This paper focuses on A*, the gold standard pathfinding algorithm in game AI. An interactive 2D grid visualizer was built using C++ and SFML to visualize A* and compare it against other foundational algorithms including Dijkstra's, Greedy Best-First, BFS, and DFS. The A* implementation is examined in depth, covering its data structures, heuristic, weighted variant, nudge tiebreaking, and snapshot storage for visualization. Although A* is not inherently parallelizable, multi-agent pathfinding is an embarrassingly parallel problem. Sequential, `std::thread`, and OpenMP strategies were compared across multiple agents, with OpenMP achieving approximately 8.46x speedup over sequential execution. Lastly, advanced A* variants such as Bidirectional A* (BA*) and Hierarchical Pathfinding A* (HPA*) are discussed as parallelizable approaches for large-scale agent scenarios.

1. INTRODUCTION

Pathfinding is fundamental to game AI. NPCs, enemies, and units all need to navigate in a 2D or 3D world. Optimal pathfinding is necessary because players can immediately notice poor AI movement. Moreover, performance is critical since pathfinding is computationally expensive in real time.

A* is the gold standard pathfinding algorithm. It is also widely used in robotics and map applications. Within game AI, A* is not only used for navigation but also for decision-making, such as searching for the lowest-cost sequence of actions in Goal-Oriented Action Planning (GOAP). Overall, A* is a general best-first search algorithm, not just a navigation tool.

In real games, the number of agents can range from tens to hundreds or even thousands. All these agents may simultaneously run pathfinding to reach the player or a target location. A naïve sequential A* approach can quickly become expensive and negatively impact game performance and player experience.

In game production, techniques such as Flow Fields, Jump Point Search, and Hierarchical Pathfinding are often used. However, these methods can become complex, especially in constantly changing environments or 3D worlds, and when handling collision avoidance and group behavior. This paper focuses on understanding A* and exploring parallelism for A*.

To demonstrate the A* pathfinding algorithm, I built an interactive 2D grid visualizer using C++, SFML, and ImGui, which will be covered in Section 2. I implemented A* in depth

using the C++ STL, explored different heuristic methods, improved A* using weights and nudging, and created animations using snapshots, which will be discussed in Section 3. To demonstrate the power of A*, I also implemented and animated other fundamental algorithms such as Dijkstra, BFS, DFS, and Greedy Best-First Search, and compared them, which will be discussed in Sections 4 and 5. After that, I explored parallelization using `std::thread` and OpenMP for multi-agent pathfinding, which will be covered in Section 6. Lastly, I will discuss my findings from research articles on two A* variants: Bidirectional A* and Hierarchical Pathfinding A*, which are highly parallelizable, in Section 7.

2. IMPLEMENTATION

2.1 Overview

C++ is the chosen language for this project, as it is the industry standard used in video game development due to its performance, as well as the language I am most comfortable with. The grid is represented as an $N \times M$ 2D array. The grid supports 4-directional movement (up, down, left, and right). The grid consists of cells, where each cell represents a node. Each node stores the following information: g-cost (distance from the start), h-cost (heuristic estimate to the goal), f-cost (total cost, where $f = g + h$), and a parent pointer, which is used to reconstruct the final path once the goal is reached.

2.2 Open and Closed Sets

The open set is implemented using `std::set` with a custom comparator based on f-cost, allowing efficient retrieval of the lowest-cost node while maintaining a unique set of nodes. The closed set is implemented using `std::unordered_set` to allow constant-time membership checks for visited nodes.

2.3 Rendering and Visualization

SFML is used to create application windows and render the grid and cell states. It also handles user interactions and input, such as selecting and deselecting start, goal, and obstacle cells.

2.4 Interaction and Controls

Users can interact with the grid through mouse input to modify the environment, including placing start and goal nodes and drawing obstacles. Additional controls are provided to run the search algorithm and reset the grid.

2.5 Snapshot-Based Animation

To animate the algorithm, the algorithm state is stored as a sequence of snapshots. Each snapshot captures the grid state at a given step, allowing the program to replay the algorithm's execution frame by frame. This approach separates computation

from rendering, making the visualization clearer and easier to control.

2.6 GUI and Configuration (ImGUI)

ImGui is used for scene control, providing options to switch between single-agent and multi-agent modes, toggle selection modes, and enable snapshot recording. It includes controls to run the search algorithm, reset the grid, and save/load grid configurations. A dropdown menu allows users to select different algorithms. For A*, additional options include enabling a tie-breaking “nudge” and adjusting the heuristic weight, which will be explained in the next section. Users can also choose between sequential execution, `std::thread`, and OpenMP. Performance metrics are displayed, including path result, path length, total nodes expanded, and search time in microseconds.

3. A* ALGORITHM

3.1 Overview

A* is a best-first search algorithm used for pathfinding. The core idea of A* is the cost function $f(n)$, which is used to determine the next cell to be explored. The cost function is defined as:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the distance from the start cell to the current cell, and $h(n)$ is the estimated distance from the current cell to the goal cell. Note that, for simplicity, the grid is designed as a uniform-cost grid where the cost between each cell is 1.

3.2 Data Structures

An open set and a closed set are used. The open set is implemented as a priority queue and stores the frontier of nodes to be explored. The priority queue ensures that the node with the lowest f -value is always processed next, representing the most promising candidate to reach the goal. The closed set is used to keep track of cells that have already been explored to avoid revisiting them.

3.3 Heuristic (Manhattan Distance)

Movement is limited to four directions; therefore, Manhattan distance is the most suitable heuristic. The formula for Manhattan distance in a 2D space is:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

where x and y represent the coordinates of two points (in this case, the row and column indices of grid cells). Manhattan distance is used as the heuristic because it is simple to compute and consistent with grid-based pathfinding where diagonal movement is not allowed.

3.4 Tie-Breaking Nudge and Weighted A*

In a uniform-cost grid, it is common for multiple cells to have the same f -cost. To improve search behavior, a small bias toward the goal is introduced using a tie-breaking nudge (typically a very small value such as 0.001).

For example, if cell A has $g = 5$, $h = 4$, and cell B has $g = 4$, $h = 5$, both have an f -cost of 9. However, applying a small scaling for the heuristic gives:

- Cell A: $f(n) = 5 + 4.004 = 9.004$
- Cell B: $f(n) = 4 + 5.005 = 9.005$

As a result, cell A is prioritized because it has a slightly lower f -cost, meaning it is marginally closer to the goal and should be explored first.

Weighted A* is another variant that introduces a faster search at the cost of optimality. It modifies the cost function as:

$$f(n) = g(n) + w \cdot h(n)$$

where $w \geq 1$ is the weight parameter. If $w = 1$, the algorithm behaves like standard A*. As the value of w increases, the search becomes faster but may produce a less optimal path due to stronger heuristic influence.

4. OTHER ALGORITHMS

4.1 Dijkstra's

Dijkstra's algorithm is a predecessor of A*. Unlike A*, Dijkstra's algorithm explores nodes based only on the distance from the start and has no knowledge of the goal location.

$$f(n) = g(n)$$

In a uniform-cost grid, Dijkstra's algorithm behaves similarly to Breadth-First Search, as both expand nodes in increasing order of distance from the start when all edge costs are equal.

4.2 Greedy Best-First

Greedy Best-First Search is a greedy variant of A* where the cost function only uses the heuristic:

$$f(n) = h(n)$$

Greedy Best-First Search is generally faster than A* but produces suboptimal paths, since it does not consider the cost already traveled. Its behavior is similar to Weighted A* when the weight value is very large (e.g., $w \gg 1$), where the heuristic dominates the search decision.

4.3 Breadth First Search

Breadth First Search (BFS) is implemented using a queue. The first node added is the first one expanded (FIFO order). BFS guarantees the shortest path in an unweighted grid. BFS explores the grid in a wave-like pattern, expanding all nodes at the current depth before moving deeper.

4.4 Depth First Search

Depth First Search (DFS) is implemented using a stack. DFS explores as far as possible in one direction before backtracking. It does not guarantee the shortest path and can become trapped in deep branches, producing a “snake-like” traversal pattern. In some cases, it may fail to find an efficient path even when the start and goal are close.

5. ALGORITHM COMPARISON

In my experiment, I tested all the algorithms on a 50×50 uniform-weighted grid. The experiment was conducted using two types of grids: an Open Grid and a Double Barrier Grid.

In the Open Grid, A* and Greedy have the lowest search time and expand the fewest nodes, with approximately 25× fewer nodes expanded. As expected, DFS produced the worst path length of 1275 nodes, compared to a path length of 99 nodes for the other algorithms. On a uniform-cost grid, Dijkstra's and BFS expand an identical number of nodes; however, Dijkstra incurs significantly higher overhead due to priority queue operations. This demonstrates that data structure choice can affect algorithm performance.

Table 1. Pathfinding algorithm performance on a 50×50 “open” grid

Algorithm	Path length	Nodes expanded	Search time (ms)
BFS	99	2500	1.53
DFS	1275	2500	1.22
Dijkstra	99	2500	4.50
A*	99	99	0.45
Greedy	99	99	0.43

In a Double Barrier Grid, we can clearly observe the advantage of A*. Although Greedy expands the fewest nodes, it produces a path length of 125 nodes, compared to A* with a path length of 99 nodes. This is due to the overestimation that results from relying only on heuristics. This is not desirable in video games, as the AI may follow inefficient or unnatural paths, which can be noticeable to players. BFS and Dijkstra also produce the optimal path, but with a larger number of nodes expanded and longer search times. DFS performs the worst, as expected.

Table 2. Pathfinding algorithm performance on a 50×50 “double barrier” grid

Algorithm	Path length	Nodes expanded	Search time (ms)
BFS	99	1614	1.14
DFS	431	857	0.65
Dijkstra	99	1594	2.70
A*	99	513	1.40
Greedy	125	442	1.24

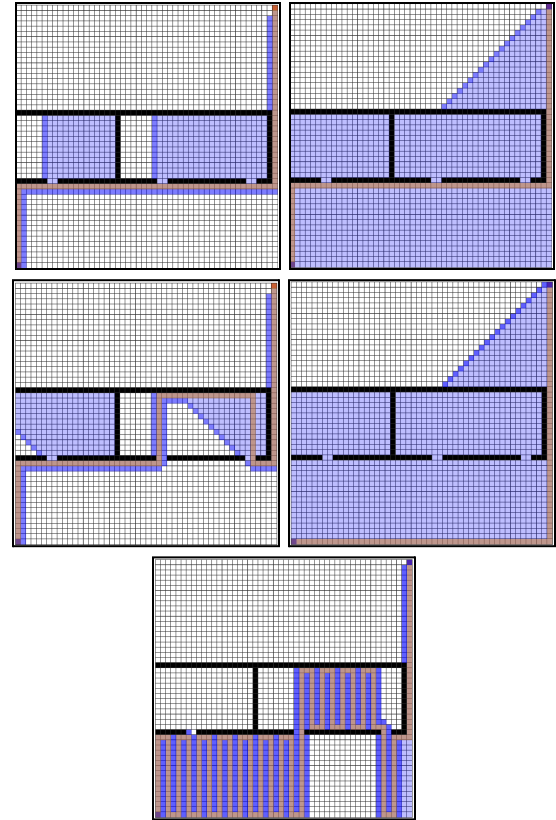


Figure 1. Algorithm Visualization on Double Barrier Grid (50×50). Left to right, top to bottom: A*, Dijkstra, Greedy, BFS, DFS. Orange = explored nodes, Yellow = final path. Start = bottom-left, Goal = top-right.

6. PARALLELIZATION

6.1 A* is not inherently parallelizable

The standard A* is not inherently parallelizable due to its reliance on a global open set (priority queue) and a closed set with a strict node expansion order. Each step in A* depends on the best node chosen from the previous steps.

6.2 Multi-agent as embarrassingly parallel

This experiment aims to test how A* handles multiple agents, simulating video games with hundreds or thousands of agents that constantly need to perform pathfinding toward a player or target, such as in real-time strategy games. Although there are specialized techniques for this scenario, such as flow fields and group movement behavior, we explore how A* performs when multiple agents execute the algorithm independently.

We also examine how this setup can be considered embarrassingly parallel. An embarrassingly parallel problem is one in which tasks are executed independently across processes or threads, with little to no need for communication between them.

In this experiment, each agent runs its own instance of the A* algorithm, maintaining its own open set, closed set, and data structures to store and look up cell information, including g, h, and f values, as well as parent nodes. These agents operate independently and do not share any data with one another.

6.3 std::thread and OpenMP

Two parallelization strategies are explored: manual threading using the C++ `std::thread` library and compiler-assisted parallelism using OpenMP

In the `std::thread` approach, a separate thread is created for each agent, where each thread executes the pathfinding algorithm independently. These threads are stored in an array and synchronized using `join()`, to ensure all agents complete their execution before the program proceeds. This approach provides explicit control over thread creation and management but introduces overhead when scaling a large number of agents due to thread lifecycle costs.

In contrast, the OpenMP approach uses a `#pragma omp parallel for` directive to automatically distribute the workload across multiple threads.

6.4 Speedup results & comparison

Table 3. A* performance comparison across sequential and parallel implementations on a 50×50 “double barrier” grid

Agents	Sequential (ms)	std::thread (ms)	OpenMP (ms)
10	13	6	4
100	107	37	31
1,000	1,070	240	190
10,000	10,750	2,000	1,270

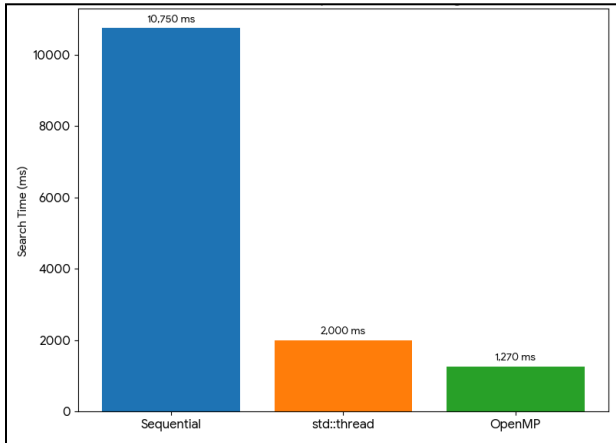


Figure 2. Total execution time comparison for a high-density (10k agent) A* workload.

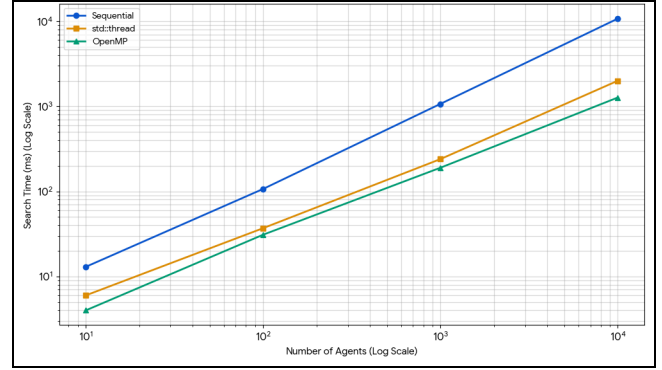


Figure 3. A* search time vs. number of agents

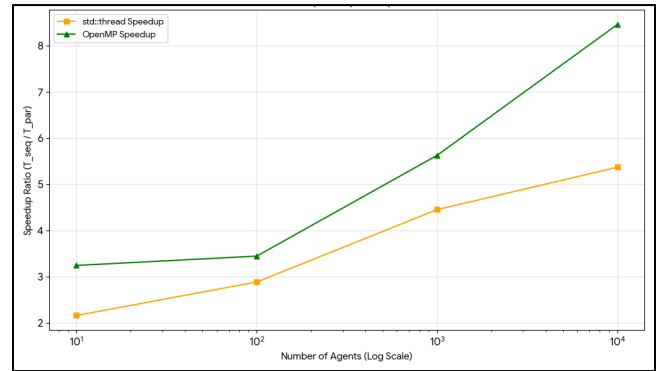


Figure 4. Speedup for parallel A* implementations relative to sequential baseline.

The experimental results demonstrate a clear performance advantage for parallel processing strategies as the computational workload increases. While the sequential implementation follows a strictly linear growth pattern, both `std::thread` and OpenMP show significant reductions in execution time. At the highest workload of 10,000 agents, the sequential A* search required 10,750 ms, whereas the `std::thread` implementation completed the same task in 2,000 ms, yielding a speedup of 5.37 times. The OpenMP implementation completed the same task in only 1,270 ms, achieving a peak speedup of approximately 8.46 times.

The main performance difference is not the speed of the A* algorithm itself, but the runtime overhead and scheduling efficiency of the threading models. In the current `std::thread` implementation, a new thread is created for every agent, which introduces a large amount of system overhead and limits performance to around 2,000 ms at the largest scale. In contrast, OpenMP uses a thread pool and work-sharing approach to reduce these costs. If a manual C++ thread pool was implemented to manage `std::thread` lifecycles, similar to how OpenMP works internally, `std::thread` would likely perform much closer to OpenMP, while also giving slightly more control over task scheduling. Overall, the results show that while parallelization is necessary for large-scale pathfinding, the choice of a low-overhead threading framework is just as important for maximizing hardware efficiency.

7. OTHER A* VARIANTS

7.1 Bidirectional A*

Bidirectional A* (BA*) is a variant of A* that runs two A* searches simultaneously, one from start to goal (forward search) and one from goal to start (backward search) [2]. At each step, we must decide which search direction to expand. There are a few alternation strategies, including round-robin or choosing the direction with the lower f-value between the two open sets [2]. The article also mentions the cardinality criterion, where the search always expands from whichever open set has fewer elements [2].

The most important aspect of BA* is the termination condition. In bidirectional search, finding a node where the two frontiers meet does not immediately guarantee an optimal path. We must keep track of the best solution found so far ($L_{\min}$) and only stop when that solution is less than or equal to the maximum of the minimum f-values across both the forward and backward open sets [2].

Bidirectional A* can technically be run in parallel, where one thread handles the forward search and another handles the backward search, with synchronization required for the termination condition. However, BA* is generally not worth the added complexity, since both searches still expand a large number of nodes. Steve Rabin in *Game AI Pro* identifies BA* as a poor choice for pathfinding optimization and instead recommends hierarchical pathfinding [3].

7.2 Hierarchical Pathfinding A*

Hierarchical Pathfinding A* (HPA*) is one of the most discussed techniques for speeding up pathfinding in large, complex, open-world maps.

In the paper "Near Optimal Hierarchical Path-Finding" by Botea et al., the goal is to reduce the complexity of searching large maps through abstraction [1]. The map is partitioned into smaller areas called clusters. Instead of searching every individual tile, the algorithm treats each cluster as a unit [1]. Entrances are the nodes along the borders of adjacent clusters, and from these entrances, transition points are created. An abstract graph is then built using these transition points and two types of edges: inter-edges, which connect nodes in adjacent clusters, and intra-edges, which represent precomputed optimal paths between nodes within the same cluster [1]. A standard A* search is then performed on this abstract graph, which is much faster than searching the full original grid.

HPA* is up to 10 times faster than a highly optimized standard A* search [1]. Moreover, in maps that change dynamically at runtime, instead of recomputing the path for the entire map, only the affected cluster needs to be updated [1]. This also scales well with multiple units, as each agent can search the abstract graph independently. Naturally, HPA* can be parallelized by processing each cluster independently during the preprocessing phase, since clusters have no dependencies on one another. At runtime, multiple agents can also run HPA* searches in parallel, combining the benefits of a reduced search space with parallel execution for large-scale game scenarios.

8. CONCLUSION

In conclusion, this paper explored A* pathfinding on a 2D grid, comparing it against the foundational algorithms and examining parallelization strategies for multi-agent scenarios. A* is the most balanced pathfinding algorithm finding the optimal path while expanding fewer nodes and having lower search times compared to BFS and Dijkstra. Although A* is not inherently parallel, multi-agent pathfinding is an embarrassingly parallel problem. Using OpenMP, we achieved a speedup of 8.46x over sequential execution with 10,000 agents. For large-scale game scenarios, HPA* is the most practical solution offering up to 10x speedup over standard A* through map abstractions and is naturally parallelizable at the cluster level.

For future work, it would be interesting to explore more on other pathfinding algorithms used in game AI, such as implementation of HPA*, Jump Point Search, Flow Fields, and navigation mesh-based approaches.

9. REFERENCES

- [1] Botea, A., Müller, M., and Schaeffer, J. Near optimal hierarchical path-finding. *Journal of Game Development* 1, 1 (2004), 7–28.
- [2] Pulido, F.J., Mandow, L., and Pérez de la Cruz, J.L. An analysis of bidirectional heuristic search in game maps. In *Proceedings of the 2011 Conference of the Spanish Association for Artificial Intelligence* (2011).
- [3] Rabin, S. and Sturtevant, N.R. Pathfinding architecture optimizations. In Steve Rabin (ed.), *Game AI Pro*. CRC Press, Boca Raton, FL, 2013, 241–252.